

can destroy all information, if, e.g., one of their linear layers has a zero weight matrix. While this is unlikely to be learned with typical pretraining, this demonstrates that if prefix-tuning could be a universal approximator, that would pose specific requirements on the pretrained MLPs and it is not clear whether real-world pretraining would satisfy these requirements.

D EXTENDED RESULTS

In this appendix we present further experiments in the context of Section 5. We consider different prefix lengths ($n_S \in \{10, 50, 100\}$) and different model sizes (4, 16 and 32 layers). We consider two extensions, the first one maintains the pretraining step as in Section 5, the other one extends the set of pretraining tasks with 4 additional tasks. Both pretraining and prefix-tuning are done for 100 000 iterations with the prefix-tuning accuracy reported every 10 000 iterations.

D.1 PRETRAINING AS IN SECTION 5

The first setting has the same pretraining tasks as in Section 5, namely sorting in ascending ($\nearrow$) or descending ($\searrow$) order, and adding one (+1) or two (+2) to each element of the input sequence. We evaluate by prefix-tuning on the same four tasks plus incrementing the ascending sorted sequence ($\nearrow+1$), double histogram ($\mathbb{H}$), element-wise modulo operation (with respect to the first element of the sequence), and FilterAtLeastNTimes which puts zeros at the positions of elements whose value appears at less than N times in the sequence with N being the first element of the sequence:

Pretraining tasks:	Sort in ascending order ($\nearrow$) Sort in descending order ($\searrow$) Add 1(+1) Add 2(+2)
Prefix-tuning tasks:	Sort in ascending order ($\nearrow$) Sort in descending order ($\searrow$) Add 1(+1) Add 2(+2) Sort ascending and add 1 ($\nearrow+1$) Modulo the first element (Modulo) Double Histogram ($\mathbb{H}$) Filter the elements that are at least as large as the first element (FilterAtLeastNTimes)

The results are plotted in Figure 9. As expected, the prefix-tuned accuracy on the pretraining tasks is close to 100%. Interestingly, prefix length 50 for the largest (32-layer) architecture appears to be an exception and does not learn the +1, $\nearrow$ and $\searrow$.

As also observed in Section 5, regardless of the prefix length and the model size, prefix-tuning a model pretrained with these four tasks cannot learn the double histogram task ($\mathbb{H}$). FilterAtLeastNTimes also appears to be challenging but the 16-layer and 32-layer experiments reach about 50% accuracy for the longest prefix size $n_S = 100$. This is curious as FilterAtLeastNTimes is related to the double histogram task: it can be considered as thresholding the double histogram output. FilterAtLeastNTimes, similarly to double histogram, is therefore not compositional in the pretraining task. It is surprising then that FilterAtLeastNTimes would achieve higher accuracy than double histogram. This hints that, perhaps, compositionality does not fully explain why prefix-tuning works for some and not other downstream tasks.

The results in Figure 9 also hint that bigger is not always better. For example, the 4-layer model prefix-tuned for the Modulo task performs better with a prefix size 50 than the larger prefix size 100. A similar effect can be observed with the 16-layer model prefix-tuned for the $\nearrow+1$ task. Larger models are also not necessarily more conducive to successful prefix-tuning: for many of the cases the 32-layer models perform worse when prefix-tuned than the 16-layer models.

D.2 EXTENDED PRETRAINING

The second setting extends the set of pretraining tasks. On top of the original three pretraining tasks $\nearrow$, $\searrow$, +1 (without +2) we also pretrain on element-wise modulo operation (with respect to the first element of the sequence), element-wise less than (less than the first element in the sequence),

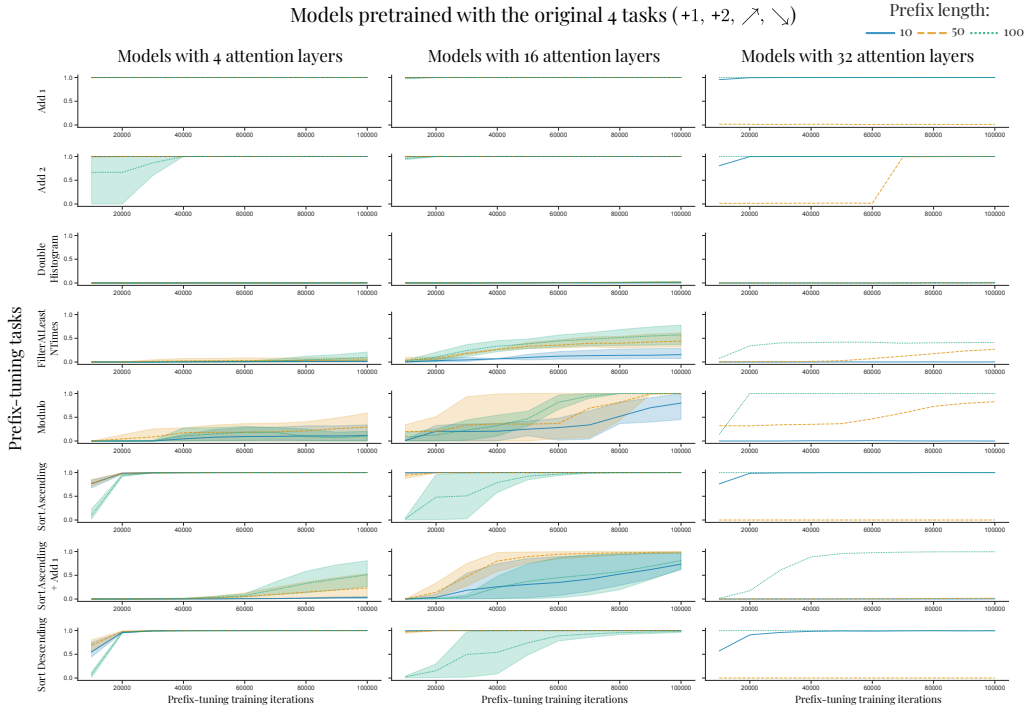


Figure 9: Extended result with pretraining as in Section 5. The pretraining and the prefix-tuning tasks are described in Appendix D.1. Each prefix was trained for 100 000 iterations and we report accuracy at every 10 000 iterations. Each experiment is performed with 3 random seeds, except the 32 layer case which, due to the computational costs involved, was performed only with one seed.

element-wise divisible (by the first element of the sequence), and inverse binary (element-wise negation). For the inverse binary task, the input is restricted to be binary. We evaluate by prefix-tuning on the same seven tasks, as well as 11 additional ones as listed in the following table:

Pretraining tasks:	Sort in ascending order ($\nearrow$)
	Sort in descending order ($\searrow$)
	Add 1 (+1)
	Modulo the first element (Modulo)
	Filter the elements that are less than the first element (LessThan)
	Filter the elements that are divisible by first element (Divisible)
	Element-wise negation (InverseBinary)
Prefix-tuning tasks:	Sort in ascending order ($\nearrow$)
	Sort in descending order ($\searrow$)
	Add 1 (+1)
	Add 2 (+2)
	Add 3 (+3)
	Modulo the first element (Modulo)
	Filter the elements that are less than the first element (LessThan)
	Filter the elements that are not less than the first element (MoreThanEqual)
	Filter the elements that are divisible by first element (Divisible)
	Filter the elements that are not divisible by first element (NotDivisible)
	Element-wise negation (InverseBinary)
	Double Histogram ($\mathbb{H}$)
	Filter the elements that are at least as large as the first element (FilterAtLeastNTimes)
	Sort ascending, followed by add 1 ($\nearrow$ +1)
	Add 1, followed by LessThan (+1 + LessThan)
	LessThan, followed by Add 1 (LessThan +1)
	LessThan, followed by sort ascending (LessThan + $\nearrow$)
	Divisible, followed by Add 1 (Divisible +1)

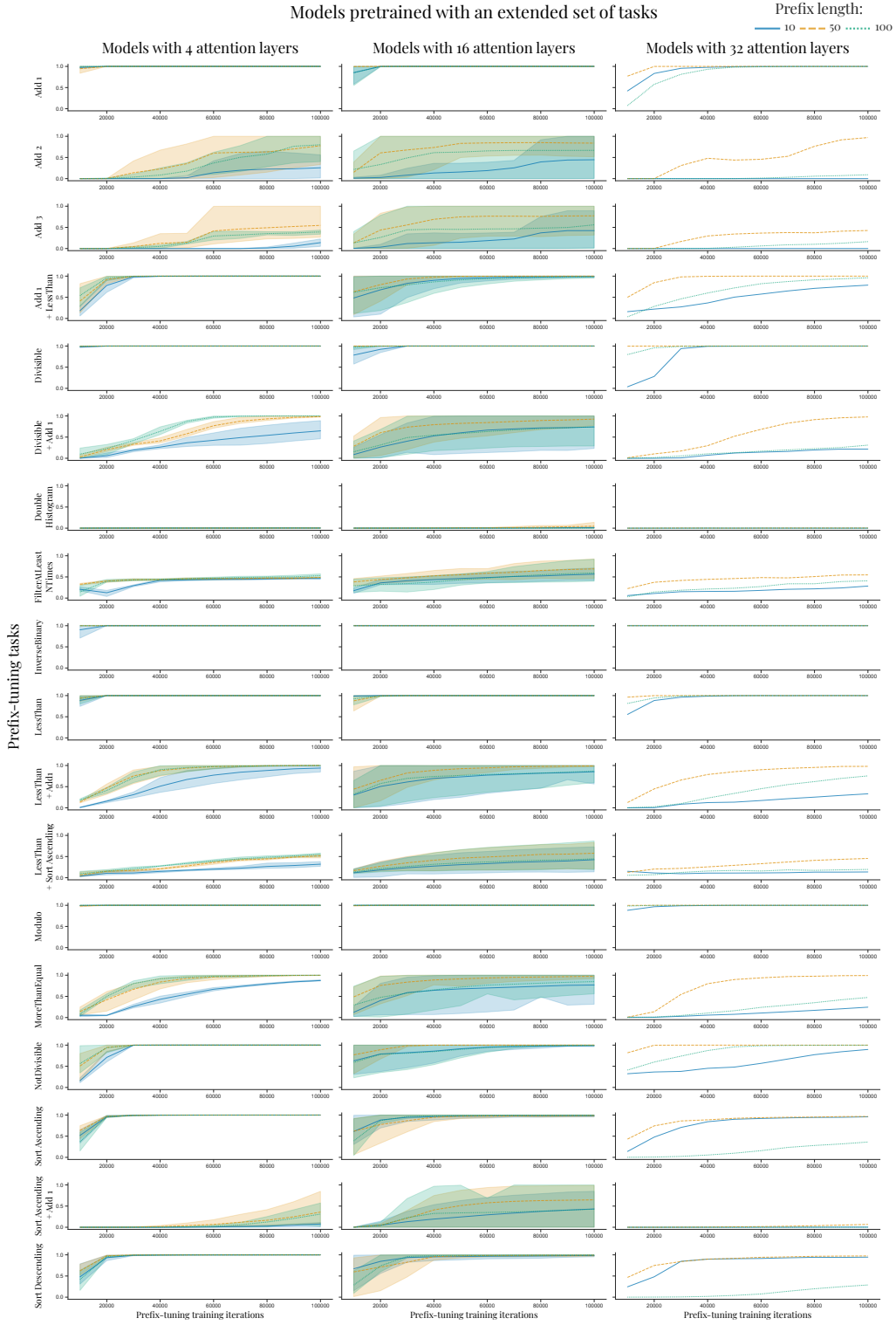


Figure 10: Extended result with pretraining with additional tasks. The pretraining and the prefix-tuning tasks are described in Appendix D.2. Each prefix was trained for 100 000 iterations and we report accuracy at every 10 000 iterations. Each experiment is performed with 3 random seeds, except the 32 layer case which, due to the computational costs involved, was performed only with one seed.